

# 計算機科学応用演習

担当者：松澤智史

提出日：2026 年 6 月 1 日

学籍番号：6324029

氏名：北澤泰樹

# 1 問題の分析

## 1.1 課題

以下の構造体と hash 関数を用い、ハッシュテーブルを構築すること

## 2 アルゴリズムの説明

### 2.1 add 関数

加えたい名前の hash 関数で番号を割り出す。malloc で必要なスペースを作成しそこに中身を書き込む書き込んだ内容をハッシュテーブルの x 番目に入れて、もともと x 番目にあったデータを next につなぐ

### 2.2 follow 関数

関数をシンプルにするために、名前（文字列）を入力するとその人のアドレスをハッシュテーブルから探し出す検索関数（find\_user）をあらかじめ作成する。

A さんが Bさんをフォローするという処理において、A と B の名前を受け取ると、まず検索関数を用いて二人の実体データ（アドレス）を特定する。その後、二人の関係性を記録するための新しい箱（follow\_burn / follow\_node 型）を malloc を用いてメモリ上に作成する。

作成した箱の「user」メンバに、フォロー相手である B さんのアドレスを代入して記録する。次に、新しく作った箱の「next」メンバに対して、それまで A さんがフォローしていた人たちのリスト（A さんの follow\_head が指していたデータ）を先に連結させる。最後に、A さんのフォローリストの先頭（follow\_head）が、今新しく作成した箱を指し示すように更新することで、リストの先頭への割り込みが完了する。

### 2.3 unfollow 関数

まず、フォローを外す側と、外される側のユーザーを特定する。次に、外す側のフォローリストを探索するため、現在地（先頭）とその一つ前を定義しておき、現在地が「外される側」になるまでリストを辿って探す。見つかった際、現在地が先頭だった場合は、フォローリストの先頭を現在地の「次」にする。それ以外のときは、現在地の「一つ前」と現在地の「次」をつなげる。最後に、列から外れて必要なくなった現在地の箱を削除（解放）する。

### 2.4 search 関数

探したい人のアドレスを関数を用いて割り出し、その人の名前と年齢を表示する。次にフォローリストは対象者のフォローリストの先頭から空になるまで書き出す。

### 2.5 is\_mutual 関数

考えたい A さん B さんのフォローリストにそれぞれが入っているか確認する

### 2.6 recommend 関数

与えられた名前からその人のアドレスを確保し、確保した A さんのフォローリストの先頭の人のフォローリストを先頭から表示する

## 2.7 find\_user 関数

探したい人の名前からハッシュ関数を使って番号（インデックス）を計算し、ハッシュテーブルの何番目の引き出しを探せばよいかを割り出す。割り出した引き出しの中からリストを先頭から順番に辿り、探している名前と一致するデータがあるか確認していく。一致する人が見つければそのデータの場所（アドレス）を返し、最後まで探しても見つからなければ「存在しない」として返す。

## 2.8 load\_data 関数

あらかじめ用意されたデータファイル（sns\_data.txt）を開き、上から順番に 1 行ずつ読み込む。ファイル内には「ユーザー情報」のブロックと「フォロー関係」のブロックが混在しているため、目印となる行（# USERS や # FOLLOWS）を探し、現在どちらのデータを読み込んでいるのかの状態を切り替えながら進める。ユーザー情報のブロックであれば、名前と年齢を読み取ってシステムに新規登録する。フォロー関係のブロックであれば、2 人の名前を読み取ってフォローの繋がりを作成する。すべての行を読み込み終わったら、ファイルを閉じる。

## 2.9 main 関数

本プログラム全体の司令塔となる関数である。まず、データを安全に管理するための準備（ハッシュテーブルの初期化）を行い、load\_data 関数を用いてテキストファイルから初期データを一括で読み込む。次に、利用可能なコマンドのメニューを画面に表示する。その後、ユーザーからの入力を待ち続ける無限ループに入る。入力された文字列を「命令（コマンド）」と「対象者（引数）」に分割し、入力された命令が「追加」「フォロー」「検索」などのどれに該当するかを判断して、対応する関数を呼び出す。終了命令（exit）が入力された場合にのみループを抜け、プログラムを終了する。

## 3 プログラムの説明

### 3.1 add 関数

まず、新しく登録するユーザーの情報を格納するため、`member_list` 型のポインタ `ptr` を宣言し、`malloc` 関数を用いて構造体のサイズ分のメモリ領域を動的に確保した。

次に、確保した領域の `age` メンバに引数として与えられた年齢を代入し、`name` メンバには `strcpy` 関数を用いて名前の文字列をコピーした。また、新規登録されたユーザーは初期状態では誰もフォローしていないため、フォローリストの先頭を表す `follow_head` には `NULL` を代入して初期化した。

続いて、ユーザーをハッシュテーブルの適切な位置に格納するため、名前からハッシュ値 `x` を算出した。最後に、ハッシュ衝突（同じハッシュ値を持つデータが複数存在する場合）に対応するため、連結リストの先頭への追加操作を行った。具体的には、`ptr` の `next` に既存の `hashtable[x]` の先頭アドレスを代入してリストを繋ぎ、`hashtable[x]` 自体のポインタを新たに作成した `ptr` に更新した。

### 3.2 follow 関数

まず、`find_user` 関数を用いて、フォロー元とフォロー先のユーザーのポインタを取得する。次に、新しいフォロー関係を格納するため、`malloc` 関数を用いて `follow_node` 型のメモリ領域を動的に確保し、その `user` メンバにフォロー先のユーザーのポインタを代入する。

リストへの追加は処理効率を考慮し、先頭への挿入として実装した。具体的には、新しく確保したノードの `next` メンバに既存のリストの先頭（`follow_head`）を代入して繋ぎ、その後、フォロー元の `follow_head` が新しいノードを指し示すようにポインタを更新している。

### 3.3 unfollow 関数

対象となる 2 名のユーザーのポインタを取得した後、フォロー元の `follow_head` から連結リストを探索する。探索の際は、現在参照しているノードを指す `current` と、一つ手前のノードを保持する `prev` の 2 つのポインタ変数を使用する。

フォロー先のユーザーを指すノードが見つかった場合、そのノードがリストの先頭（`current == follow_head`）であれば、`follow_head` を `current->next` に更新する。先頭以外であれば、`prev->next` に `current->next` を代入することで対象ノードをリストから切り離す。最後に、メモリーリークを防ぐため、不要となったノードの領域を `free` 関数を用いて OS に解放する。

### 3.4 search 関数

検索対象ユーザーのポインタを `find_user` 関数で取得し、構造体の `name` および `age` メンバを参照してプロフィールを標準出力する。

続いて、対象ユーザーの `follow_head` から始まる連結リストを、`while` ループを用いて探索する。`current` ポインタが `NULL` になるまでリストを順に辿り、各ノードの `user` メンバ（フォロー相手の構造体へのポインタ）を経由して、相手の `name` メンバを取得し順次出力することで、フォローリストの一覧表示を実現している。

### 3.5 is\_mutual 関数

2 人のユーザーが相互にフォローしている状態（有向グラフにおける双方向のエッジ）を判定する。双方のポインタを取得後、それぞれの方向でフォロー関係が存在するかを記録するフラグ変数を用意する。

一方のユーザーのフォローリストを while ループで探索し、各ノードの user メンバが相手のポインタと一致するかを == 演算子で比較する。文字列比較ではなくアドレス同士を直接比較することで判定処理を効率化している。相手が見つければフラグを立ててループを抜け、逆方向についても同様の探索を行う。最終的に両方のフラグが立っている場合のみ、相互フォロー状態と判定して結果を出力する。

### 3.6 recommend 関数

対象ユーザーに対して「友人の友人（有向グラフにおける距離 2 のノード）」を推薦する機能である。対象ユーザーのフォローリストを外側の while ループで辿り（友人）、さらに各友人のフォローリストを内側の while ループで辿る（友人の友人）という 2 重ループ構造を用いて探索を行う。

出力の際、推薦対象となるユーザーのアドレス（current\_fof->user）が、検索の起点となったユーザー自身のアドレス（target）と一致しない場合のみ出力処理を行う条件分岐を設けることで、自分自身が推薦リストに含まれることを防いでいる。

### 3.7 find\_user 関数

引数として与えられた検索対象の名前（name）から hash 関数を用いてインデックス（index）を計算し、ハッシュテーブルの該当箇所（hashtable[index]）の先頭アドレスをポインタ変数 current に代入する。

ハッシュ衝突によって複数のデータが同じインデックスに格納されている場合を考慮し、while ループを用いて current が NULL になるまで連結リストを順に探索する。ループ内では strcmp 関数を用いて current の name メンバと引数の name を比較する。文字列が完全に一致（戻り値が 0）した場合、目的のユーザーの実体データへのポインタ（current）を返す。リストの末尾まで探索しても一致するノードが存在しなかった場合は、該当ユーザーがないことを示す NULL を返す。

### 3.8 load\_data 関数

fopen 関数を用いて、引数で指定されたファイルを読み込みモード（"r"）で開く。fgets 関数を用いて while ループで 1 行ずつ文字列として読み込み、strcspn 関数を用いて末尾の改行文字（\n）を削除して整形する。読み込んだ行がどのデータに該当するかを判別するため、状態を管理する mode 変数（1: USERS, 2: FOLLOWS）を用いる。行の先頭が '#' の場合は、strstr 関数で "# USERS" または "# FOLLOWS" を検知して mode を切り替える。mode が 1 の場合は、sscanf 関数で文字列から名前（文字列）と年齢（整数）を抽出し、add 関数を呼び出してユーザーを登録する。mode が 2 の場合も同様に sscanf で 2 名の名前を抽出し、ポインタの繋ぎ替えを行ってフォロー関係を構築する。最後に fclose 関数でファイルを安全に閉じる。

### 3.9 main 関数

プログラムの起点である。まず、予期せぬメモリエラーを防ぐため、ハッシュテーブル (hashtable 配列) の全要素を for 文を用いて NULL で初期化する。その後、load\_data 関数と print\_menu 関数を呼び出し、初期設定と画面表示を行う。

対話的なコマンド受付は、while(1) を用いた無限ループで実装している。ループ内では fgets 関数を用いて標準入力 (stdin) から入力を受け取る (EOF が入力された場合は break で安全に終了する)。入力された文字列は、sscanf 関数を用いてコマンド部分 (cmd) と、それに続く最大 2 つの引数 (arg1, arg2) に分割する。その後、strcmp 関数を用いて cmd の文字列を判定し、if-else if 構文で分岐させて各機能の関数を呼び出す。この際、add 関数の年齢引数など、数値が必要な場合は atoi 関数を用いて文字列を整数に変換して渡している。"exit" が入力されると break でループを脱出し、return 0 で正常終了する。

## 4 実行結果とその評価

### 4.1 実行結果

```
bash-4.4$ a.out
Loading data from sns_data.txt... Done.
--- User Management System ---
Commands:
search [name] : Show user info and following list ...

command > search
Unknown command or missing arguments.

command > search sato_taro
[User] sato_taro (64)
Following: murakami_naoko okada_yuma hashimoto_naoki kimura_mami

command > follow sato_taro ogawa_rina
Success: sato_taro followed ogawa_rina.

command > is_mutual sato_taro hashimoto_naoki
Yes, sato_taro and hashimoto_naoki are mutual friends!

command > unfollow sato_taro ogawa_rina
Success: sato_taro unfollowed ogawa_rina.

command > recommend sato_taro
Recommended for sato_taro:
- okamoto_satoshi (because murakami_naoko follows them)
- okada_hanako (because murakami_naoko follows them)
- nishimura_aoi (because okada_yuma follows them)
- nakamura_naoki (because okada_yuma follows them)
- takahashi_saburo (because hashimoto_naoki follows them)
- sakamoto_nanami (because hashimoto_naoki follows them)
- hayashi_tatsuya (because hashimoto_naoki follows them)
- ono_mai (because kimura_mami follows them)
- ono_shiro (because kimura_mami follows them)
- suzuki_yuta (because kimura_mami follows them)
- nakano_erika (because kimura_mami follows them)
- itoh_ryo (because kimura_mami follows them)

command > exit
Goodbye!
```

図 1

## 5 考察

### 5.1 おすすめユーザー推薦機能の改善と実装

本システムで実装した `recommend` 関数は、対象ユーザーの「友人の友人（深さ 2 のノード）」を検索し、自分自身を除外して推薦リストを表示する仕様とした。しかし、実用的な SNS の観点からは「すでに自分がフォローしているユーザー」が推薦されてしまうのは冗長である。

この問題を解決するためには、推薦候補 (`current_fof->user`) が見つかった際に、それが「対象ユーザー (`target`)」の `follow_head` から始まるリスト内に存在しないかを確認する条件分岐を追加する必要がある。これにより、真の意味で「まだフォローしていない新たな繋がり」のみを提示できるようになり、システムの実用性が大きく向上すると考えられる。

そこで、本課題の発展として実際にこの改良を施した関数を実装した。新たに作成した `is_following` は、あるユーザーが特定の相手をすでにフォローしているかを判定する補助関数である。内部の探索ロジックは先述の `is_mutual` 関数とほぼ同様であるため、詳細な説明は割愛する。既存の `recommend` 関数からの主な変更点は、出力判定を行う `if` 文の条件式に論理積 (`&&`) を用い、`is_following(target, current_fof->user) == 0` を追加したことである。改良したコードを以下に示す。

Listing 1: 改良版 `recommend` 関数と `is_following` 関数

```
1 int is_following(struct member_list *from, struct member_list *to) {
2     struct follow_node *current = from->follow_head;
3
4     // 自身のフォローリストを順に確認
5     while (current != NULL) {
6         if (current->user == to) {
7             return 1; // すでにフォローしている (True)
8         }
9         current = current->next;
10    }
11    return 0; // フォローしていない (False)
12 }
13
14 void recommend(char *name) {
15     struct member_list *target = find_user(name);
16     if (target == NULL) {
17         printf("Error: User not found.\n");
18         return;
19    }
20
21    printf("Recommended for %s:\n", name);
22    struct follow_node *current_friend = target->follow_head;
23
24    while (current_friend != NULL) {
25        struct follow_node *current_fof = current_friend->user->follow_head;
26        while (current_fof != NULL) {
27            // 自身ではなく、かつ、まだフォローしていない場合のみ表示
```

```

28         if (current_fof->user != target && is_following(target, current_fof
29             ->user) == 0) {
30             printf("-_s_(because_s_follows_them)\n", current_fof->user->
31                 name, current_friend->user->name);
32         }
33         current_fof = current_fof->next;
34     }
35     current_friend = current_friend->next;
36 }

```

## 5.2 ハッシュ関数の偏りと性能調査

### 5.2.1 dist 関数の作成

```

1 void dist() {
2     printf("--- Hash Distribution ---\n");
3
4     int total_users = 0; // 全ユーザー数
5
6     // テーブルの端から端まで (0 から TABLE_SIZE - 1 まで) 確認する
7     for (int i = 0; i < TABLE_SIZE; i++) {
8
9         int count = 0; // このバケットに入っている人数
10        struct member_list *current = hashtable[i];
11
12        while(current != NULL){
13            current = current->next;
14            count++;
15        }
16
17
18
19        // もし人以上入っていれば、そのバケットの人数を表示する1
20        if (count > 0) {
21            printf("Bucket [%3d]: %d users\n", i, count);
22        }
23
24        total_users += count; // 全体の人数に足し合わせる
25    }
26
27    printf("-----\n");
28    printf("Total users loaded: %d\n", total_users);
29
30    // 理想的な平均人数を計算して表示する
31    // (全体の人数÷ハッシュテーブルのサイズ)
32    double average = (double)total_users / TABLE_SIZE;
33    printf("Ideal average per bucket: %.2f users\n", average);

```



### 5.2.2 9万のデータを入れた結果

```
command > dist
--- Hash Distribution ---
Bucket [ 0]: 5283 users
Bucket [ 1]: 5460 users
Bucket [ 2]: 5520 users
Bucket [ 3]: 5460 users
Bucket [ 4]: 5283 users
Bucket [ 5]: 4998 users
Bucket [ 6]: 4620 users
Bucket [ 7]: 4170 users
Bucket [ 8]: 3675 users
Bucket [ 9]: 3162 users
Bucket [10]: 2654 users
Bucket [11]: 2170 users
Bucket [12]: 1725 users
Bucket [13]: 1330 users
Bucket [14]: 992 users
Bucket [15]: 714 users
Bucket [16]: 495 users
Bucket [17]: 330 users
Bucket [18]: 210 users
Bucket [19]: 126 users
Bucket [20]: 70 users
Bucket [21]: 35 users
Bucket [22]: 15 users
Bucket [23]: 5 users
Bucket [24]: 1 users
Bucket [82]: 5 users
Bucket [83]: 15 users
Bucket [84]: 35 users
Bucket [85]: 70 users
Bucket [86]: 126 users
Bucket [87]: 210 users
Bucket [88]: 330 users
Bucket [89]: 495 users
Bucket [90]: 715 users
Bucket [91]: 992 users
Bucket [92]: 1330 users
Bucket [93]: 1725 users
Bucket [94]: 2170 users
Bucket [92]: 1330 users
Bucket [93]: 1725 users
Bucket [94]: 2170 users
Bucket [95]: 2654 users
Bucket [96]: 3162 users
Bucket [97]: 3675 users
Bucket [98]: 4170 users
Bucket [99]: 4620 users
Bucket [100]: 4998 users
-----
Total users loaded: 90000
Ideal average per bucket: 891.09 users
command > 
```

図 2: 改善前のハッシュ分布 (9 万件のデータ入力時)

### 5.2.3 現状の考察

図 1,2 からわかるように 24 のバケットには一人しかいないところと 5000 人近く同じバケットにいる。また、表示されないバケットも多く存在する。

## 5.3 衝突が少ないハッシュ関数への改良

### 5.3.1 改善前のハッシュ関数

```
1 // ハッシュ関数
2 int hash(char *key) {
3     unsigned int hashval = 0;
4     while (*key != '\0') {
5         hashval += *key;
6         key++;
7     }
8     return hashval % TABLE_SIZE;
9 }
```

### 5.3.2 改善後のハッシュ関数

```
1 // 改良版ハッシュ関数 (アルゴリズム) DJB2
2 int hash(char *key) {
3     unsigned long hashval = 5381;
4     int c;
5
6     while ((c = (unsigned char)*key++)) {
7         hashval = ((hashval << 5) + hashval) + c; // hashval * 33 + c
8     }
9     return (int)(hashval % TABLE_SIZE);
10 }
```

### 5.3.3 説明

従来のハッシュ関数は文字列の ASCII コードを単純に足し合わせをおこなっていた。改善後は DJB2 アルゴリズムを採用した。このアルゴリズムは初期値 5381 に対し、文字列の各文字列を読み込むごとに「現在のハッシュ値を 33 倍し、文字コードを足す」という処理を繰り返す。文字の並び順やわずかな文字の違いが最終的なハッシュ値に大きく反映されるため、ハッシュテーブル全体へデータが均等に分散され、検索の最悪時間計算量が  $O(N)$  に肥大化するのを防ぐ効果がある。

### 5.3.4 9万のデータを入れた結果

```
command > dist
--- Hash Distribution ---
Bucket [ 0]: 8 users
Bucket [ 1]: 12 users
Bucket [ 2]: 10 users
Bucket [ 3]: 5 users
Bucket [ 4]: 16 users
Bucket [ 5]: 17 users
Bucket [ 6]: 10 users
Bucket [ 7]: 7 users
Bucket [ 8]: 13 users
Bucket [ 9]: 8 users
Bucket [10]: 6 users
Bucket [11]: 10 users
Bucket [12]: 11 users
Bucket [13]: 15 users
Bucket [14]: 8 users
Bucket [15]: 5 users
Bucket [16]: 8 users
Bucket [17]: 12 users
Bucket [18]: 7 users
Bucket [19]: 9 users
Bucket [20]: 15 users
Bucket [21]: 15 users
Bucket [22]: 7 users
Bucket [23]: 8 users
Bucket [24]: 15 users
Bucket [25]: 13 users
Bucket [26]: 15 users
Bucket [27]: 5 users
Bucket [28]: 7 users
Bucket [29]: 15 users
Bucket [30]: 7 users
Bucket [31]: 8 users
Bucket [32]: 9 users
Bucket [33]: 9 users
Bucket [34]: 10 users
Bucket [35]: 8 users
Bucket [36]: 14 users
Bucket [37]: 8 users
Bucket [38]: 10 users
Bucket [39]: 10 users
Bucket [40]: 5 users
Bucket [41]: 14 users
Bucket [42]: 14 users
Bucket [43]: 7 users
Bucket [44]: 10 users
Bucket [45]: 7 users
Bucket [46]: 8 users
Bucket [47]: 15 users
Bucket [48]: 8 users
Bucket [49]: 7 users
Bucket [50]: 7 users
Bucket [51]: 14 users
Bucket [52]: 10 users
Bucket [53]: 8 users
Bucket [54]: 14 users
Bucket [55]: 6 users
Bucket [56]: 19 users
Bucket [57]: 15 users
Bucket [58]: 7 users
Bucket [59]: 12 users
Bucket [60]: 7 users
Bucket [61]: 11 users
Bucket [62]: 7 users
Bucket [63]: 10 users
Bucket [64]: 6 users
Bucket [65]: 6 users
Bucket [66]: 11 users
Bucket [67]: 18 users
Bucket [68]: 9 users
Bucket [69]: 9 users
Bucket [70]: 9 users
Bucket [71]: 10 users
Bucket [72]: 13 users
Bucket [73]: 5 users
Bucket [74]: 12 users
Bucket [75]: 3 users
Bucket [76]: 8 users
Bucket [77]: 11 users
Bucket [78]: 9 users
Bucket [79]: 10 users
Bucket [80]: 17 users
Bucket [81]: 8 users
Bucket [82]: 11 users
Bucket [83]: 10 users
Bucket [84]: 8 users
Bucket [85]: 9 users
Bucket [86]: 12 users
Bucket [87]: 10 users
Bucket [88]: 3 users
Bucket [89]: 12 users
Bucket [90]: 10 users
Bucket [91]: 8 users
Bucket [92]: 6 users
Bucket [93]: 9 users
Bucket [94]: 6 users
Bucket [95]: 11 users
Bucket [96]: 9 users
Bucket [97]: 18 users
Bucket [98]: 4 users
Bucket [99]: 13 users
Bucket [100]: 9 users
-----
Total users loaded: 1000
Ideal average per bucket: 9.90 users
command > []
```

図 3: 改善後のハッシュ分布 (9 万件のデータ入力時)

#### 5.3.5 結果の考察

すべてのバケットに入っている user 数は 9.90 前後に抑えられている。また、表示されていないバケットはほとんどない。以上より改善が見られた。

## 6 ソースコード

### 6.1 SNS データ管理 改良前

Listing 2: ソースコード

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define TABLE_SIZE 101
6
7 struct member_list {
8     char name[64];
9     int age;
10    struct follow_node *follow_head;
11    struct member_list *next;
12 };
13
14 struct follow_node {
15     struct member_list *user;
16     struct follow_node *next;
17 };
18
19 struct member_list *hashtable[TABLE_SIZE];
20
21 // ハッシュ関数
22 int hash(char *key) {
23     unsigned int hashval = 0;
24     while (*key != '\0') {
25         hashval += *key;
26         key++;
27     }
28     return hashval % TABLE_SIZE;
29 }
30
31 // ユーザー追加
32 void add(char *name, int age) {
33     struct member_list *ptr;
34     int x = hash(name);
35     ptr = malloc(sizeof(struct member_list));
36     if (ptr == NULL) {
37         printf("エラー\n");
38         return;
39     }
40
41     ptr->age = age;
```

```

42     strcpy(ptr->name, name);
43     ptr->follow_head = NULL;
44
45     ptr->next = hashtable[x];
46     hashtable[x] = ptr;
47 }
48
49 // 人を探す
50 struct member_list *find_user(char *name) {
51     int index = hash(name);
52     struct member_list *current = hashtable[index];
53
54     while (current != NULL) {
55         if (strcmp(current->name, name) == 0) {
56             return current;
57         }
58         current = current->next;
59     }
60     return NULL;
61 }
62
63 // フォローする
64 void follow(char *from, char *to) {
65     struct member_list *from_user = find_user(from);
66     struct member_list *to_user = find_user(to);
67
68     if (from_user == NULL || to_user == NULL) {
69         printf("Error: User not found.\n");
70         return;
71     }
72
73     struct follow_node *new_follow = malloc(sizeof(struct follow_node));
74     if (new_follow == NULL) {
75         printf("エラー\n");
76         return;
77     }
78
79     new_follow->user = to_user;
80     new_follow->next = from_user->follow_head;
81     from_user->follow_head = new_follow;
82
83     printf("Success: %s followed %s.\n", from, to);
84 }
85
86 // フォロー解除
87 void unfollow(char *from, char *to) {
88     struct member_list *from_user = find_user(from);
89     struct member_list *to_user = find_user(to);
90

```

```

91     if (from_user == NULL || to_user == NULL) {
92         printf("Error: ユーザーが見つかりません.\n");
93         return;
94     }
95
96     struct follow_node *current = from_user->follow_head;
97     struct follow_node *prev = NULL;
98
99     while (current != NULL) {
100         if (current->user == to_user) {
101             break;
102         }
103         prev = current;
104         current = current->next;
105     }
106
107     if (current == NULL) {
108         printf("Error: フォローしていないユーザーです.\n");
109         return;
110     }
111
112     if (current == from_user->follow_head) {
113         from_user->follow_head = current->next;
114     } else {
115         prev->next = current->next;
116     }
117
118     free(current);
119     printf("Success: ユーザー %s は %s をフォロー解除しました.\n", from, to);
120 }
121
122 // ユーザー情報とフォローリストの表示
123 void search(char *name) {
124     struct member_list *target = find_user(name);
125
126     if (target == NULL) {
127         printf("Error: ユーザーが見つかりません.\n");
128         return;
129     }
130
131     printf("[User] %s (%d)\n", target->name, target->age);
132     printf("Following: ");
133
134     struct follow_node *current = target->follow_head;
135     while (current != NULL) {
136         printf("%s ", current->user->name);
137         current = current->next;
138     }
139     printf("\n");

```

```

140 }
141
142 // 相互フォロー判定
143 void is_mutual(char *name1, char *name2) {
144     struct member_list *user1 = find_user(name1);
145     struct member_list *user2 = find_user(name2);
146
147     if (user1 == NULL || user2 == NULL) {
148         printf("Error: User not found.\n");
149         return;
150     }
151
152     int follow1_to_2 = 0;
153     int follow2_to_1 = 0;
154
155     struct follow_node *current1 = user1->follow_head;
156     while (current1 != NULL) {
157         if (current1->user == user2) {
158             follow1_to_2 = 1;
159             break;
160         }
161         current1 = current1->next;
162     }
163
164     struct follow_node *current2 = user2->follow_head;
165     while (current2 != NULL) {
166         if (current2->user == user1) {
167             follow2_to_1 = 1;
168             break;
169         }
170         current2 = current2->next;
171     }
172
173     if (follow1_to_2 == 1 && follow2_to_1 == 1) {
174         printf("Yes, %s and %s are mutual friends!\n", name1, name2);
175     } else {
176         printf("No, %s and %s are not mutual friends.\n", name1, name2);
177     }
178 }
179
180 // おすすめユーザー推薦
181 void recommend(char *name) {
182     struct member_list *target = find_user(name);
183     if (target == NULL) {
184         printf("Error: User not found.\n");
185         return;
186     }
187
188     printf("Recommended for %s:\n", name);

```

```

189     struct follow_node *current_friend = target->follow_head;
190
191     while (current_friend != NULL) {
192         struct follow_node *current_fof = current_friend->user->follow_head;
193         while (current_fof != NULL) {
194             if (current_fof->user != target) {
195                 printf("-_s_(because_s_follows_them)\n", current_fof->user->
196                     name, current_friend->user->name);
197             }
198             current_fof = current_fof->next;
199         }
200         current_friend = current_friend->next;
201     }
202
203 // ファイル読み込み処理
204 void load_data(char *filename) {
205     FILE *fp = fopen(filename, "r");
206     if (fp == NULL) {
207         printf("Error:_Cannot_open_file_s\n", filename);
208         return;
209     }
210
211     char line[256];
212     int mode = 0; // 0: 初期, 1: USERS, 2: FOLLOWS
213
214     while (fgets(line, sizeof(line), fp) != NULL) {
215         line[strcspn(line, "\n")] = 0;
216
217         if (line[0] == '#' || line[0] == '\\0') {
218             if (strstr(line, "#_USERS")) mode = 1;
219             else if (strstr(line, "#_FOLLOWS")) mode = 2;
220             continue;
221         }
222
223         if (mode == 1) {
224             char name[64];
225             int age;
226             if (sscanf(line, "%s_d", name, &age) == 2) {
227                 add(name, age);
228             }
229         } else if (mode == 2) {
230             char from[64], to[64];
231             if (sscanf(line, "%s_s", from, to) == 2) {
232                 // 初期読み込み時は "Success" の表示を消すために静かに繋ぐ
233                 struct member_list *from_user = find_user(from);
234                 struct member_list *to_user = find_user(to);
235                 if (from_user && to_user) {

```

```

236         struct follow_node *new_follow = malloc(sizeof(struct
                follow_node));
237         new_follow->user = to_user;
238         new_follow->next = from_user->follow_head;
239         from_user->follow_head = new_follow;
240     }
241 }
242 }
243 }
244 fclose(fp);
245 }
246
247 // メニュー表示
248 void print_menu() {
249     printf("---User Management System---\n");
250     printf("Commands:\n");
251     printf("search [name] : Show user info and following list\n");
252     printf("add [name] [age] : Register a new user\n");
253     printf("follow [from] [to] : Create a follow relationship\n");
254     printf("unfollow [from] [to] : Remove a follow relationship\n");
255     printf("is_mutual [name1] [name2] : Check mutual follow status\n");
256     printf("recommend [name] : Show 'friend of friends'\n");
257     printf("exit : Quit program\n");
258     printf("-----\n\n");
259 }
260
261 // 関数 main
262 int main() {
263     char input[256];
264     char cmd[32], arg1[64], arg2[64];
265
266     // 配列の初期化安全のため ()
267     for (int i = 0; i < TABLE_SIZE; i++) {
268         hashtable[i] = NULL;
269     }
270
271     // 1. データの読み込み
272     printf("Loading data from sns_data.txt...\n");
273     load_data("sns_data.txt");
274     printf("Done.\n");
275
276     // 2. メニューの表示
277     print_menu();
278
279     // 3. 対話的なコマンド受付ループ
280     while (1) {
281         printf("command>");
282
283         if (fgets(input, sizeof(input), stdin) == NULL) {

```

```

284         break;
285     }
286     input[strcspn(input, "\n")] = '\0';
287     if (strlen(input) == 0) continue;
288
289     cmd[0] = '\0'; arg1[0] = '\0'; arg2[0] = '\0';
290     int num_args = sscanf(input, "%s%s%s", cmd, arg1, arg2);
291
292     if (strcmp(cmd, "exit") == 0) {
293         printf("Goodbye!\n");
294         break;
295     }
296     else if (strcmp(cmd, "search") == 0 && num_args >= 2) {
297         search(arg1);
298     }
299     else if (strcmp(cmd, "add") == 0 && num_args >= 3) {
300         add(arg1, atoi(arg2));
301         printf("Success: Added %s.\n", arg1);
302     }
303     else if (strcmp(cmd, "follow") == 0 && num_args >= 3) {
304         follow(arg1, arg2);
305     }
306     else if (strcmp(cmd, "unfollow") == 0 && num_args >= 3) {
307         unfollow(arg1, arg2);
308     }
309     else if (strcmp(cmd, "is_mutual") == 0 && num_args >= 3) {
310         is_mutual(arg1, arg2);
311     }
312     else if (strcmp(cmd, "recommend") == 0 && num_args >= 2) {
313         recommend(arg1);
314     }
315     else {
316         printf("Unknown command or missing arguments.\n");
317     }
318
319     printf("\n");
320 }
321
322 return 0;
323 }

```

## 6.2 recommend 関数改良後、ハッシュ関数改良前

Listing 3: ソースコード

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>

```

```

4
5 #define TABLE_SIZE 101
6
7 struct member_list {
8     char name[64];
9     int age;
10    struct follow_node *follow_head;
11    struct member_list *next;
12 };
13
14 struct follow_node {
15     struct member_list *user;
16     struct follow_node *next;
17 };
18
19 struct member_list *hashtable[TABLE_SIZE];
20
21 // ハッシュ関数
22 int hash(char *key) {
23     unsigned int hashval = 0;
24     while (*key != '\0') {
25         hashval += *key;
26         key++;
27     }
28     return hashval % TABLE_SIZE;
29 }
30
31 // ユーザー追加
32 void add(char *name, int age) {
33     struct member_list *ptr;
34     int x = hash(name);
35     ptr = malloc(sizeof(struct member_list));
36     if (ptr == NULL) {
37         printf("エラー\n");
38         return;
39     }
40
41     ptr->age = age;
42     strcpy(ptr->name, name);
43     ptr->follow_head = NULL;
44
45     ptr->next = hashtable[x];
46     hashtable[x] = ptr;
47 }
48
49 // 人を探す
50 struct member_list *find_user(char *name) {
51     int index = hash(name);
52     struct member_list *current = hashtable[index];

```

```

53
54 while (current != NULL) {
55     if (strcmp(current->name, name) == 0) {
56         return current;
57     }
58     current = current->next;
59 }
60 return NULL;
61 }
62
63 // フォローする
64 void follow(char *from, char *to) {
65     struct member_list *from_user = find_user(from);
66     struct member_list *to_user = find_user(to);
67
68     if (from_user == NULL || to_user == NULL) {
69         printf("Error: ユーザーが見つかりません.\n");
70         return;
71     }
72
73     struct follow_node *new_follow = malloc(sizeof(struct follow_node));
74     if (new_follow == NULL) {
75         printf("エラー\n");
76         return;
77     }
78
79     new_follow->user = to_user;
80     new_follow->next = from_user->follow_head;
81     from_user->follow_head = new_follow;
82
83     printf("Success: %sが%sをフォローしました.\n", from, to);
84 }
85
86 // フォロー解除
87 void unfollow(char *from, char *to) {
88     struct member_list *from_user = find_user(from);
89     struct member_list *to_user = find_user(to);
90
91     if (from_user == NULL || to_user == NULL) {
92         printf("Error: ユーザーが見つかりません.\n");
93         return;
94     }
95
96     struct follow_node *current = from_user->follow_head;
97     struct follow_node *prev = NULL;
98
99     while (current != NULL) {
100         if (current->user == to_user) {
101             break;

```

```

102     }
103     prev = current;
104     current = current->next;
105 }
106
107 if (current == NULL) {
108     printf("Error: Not following.\n");
109     return;
110 }
111
112 if (current == from_user->follow_head) {
113     from_user->follow_head = current->next;
114 } else {
115     prev->next = current->next;
116 }
117
118 free(current);
119 printf("Success: %s unfollowed %s.\n", from, to);
120 }
121
122 // ユーザー情報とフォローリストの表示
123 void search(char *name) {
124     struct member_list *target = find_user(name);
125
126     if (target == NULL) {
127         printf("Error: User not found.\n");
128         return;
129     }
130
131     printf("[User] %s (%d)\n", target->name, target->age);
132     printf("Following:");
133
134     struct follow_node *current = target->follow_head;
135     while (current != NULL) {
136         printf("%s", current->user->name);
137         current = current->next;
138     }
139     printf("\n");
140 }
141
142 // 相互フォロー判定
143 void is_mutual(char *name1, char *name2) {
144     struct member_list *user1 = find_user(name1);
145     struct member_list *user2 = find_user(name2);
146
147     if (user1 == NULL || user2 == NULL) {
148         printf("Error: User not found.\n");
149         return;
150     }

```

```

151
152     int follow1_to_2 = 0;
153     int follow2_to_1 = 0;
154
155     struct follow_node *current1 = user1->follow_head;
156     while (current1 != NULL) {
157         if (current1->user == user2) {
158             follow1_to_2 = 1;
159             break;
160         }
161         current1 = current1->next;
162     }
163
164     struct follow_node *current2 = user2->follow_head;
165     while (current2 != NULL) {
166         if (current2->user == user1) {
167             follow2_to_1 = 1;
168             break;
169         }
170         current2 = current2->next;
171     }
172
173     if (follow1_to_2 == 1 && follow2_to_1 == 1) {
174         printf("Yes, %s and %s are mutual friends!\n", name1, name2);
175     } else {
176         printf("No, %s and %s are not mutual friends.\n", name1, name2);
177     }
178 }
179
180 // おすすめユーザー推薦
181 void recommend(char *name) {
182     struct member_list *target = find_user(name);
183     if (target == NULL) {
184         printf("Error: User not found.\n");
185         return;
186     }
187
188     printf("Recommended for %s:\n", name);
189     struct follow_node *current_friend = target->follow_head;
190
191     while (current_friend != NULL) {
192         struct follow_node *current_fof = current_friend->user->follow_head;
193         while (current_fof != NULL) {
194             if (current_fof->user != target) {
195                 printf("- %s (because %s follows them)\n", current_fof->user->
196                     name, current_friend->user->name);
197             }
198             current_fof = current_fof->next;
199         }
200         current_friend = current_friend->next;
201     }

```

```

199         current_friend = current_friend->next;
200     }
201 }
202
203 // ファイル読み込み処理
204 void load_data(char *filename) {
205     FILE *fp = fopen(filename, "r");
206     if (fp == NULL) {
207         printf("Error: Cannot open file %s\n", filename);
208         return;
209     }
210
211     char line[256];
212     int mode = 0; // 0: 初期, 1: USERS, 2: FOLLOWS
213
214     while (fgets(line, sizeof(line), fp) != NULL) {
215         line[strcspn(line, "\n")] = 0;
216
217         if (line[0] == '#' || line[0] == '\\0') {
218             if (strstr(line, "#_USERS")) mode = 1;
219             else if (strstr(line, "#_FOLLOWS")) mode = 2;
220             continue;
221         }
222
223         if (mode == 1) {
224             char name[64];
225             int age;
226             if (sscanf(line, "%s_%d", name, &age) == 2) {
227                 add(name, age);
228             }
229         } else if (mode == 2) {
230             char from[64], to[64];
231             if (sscanf(line, "%s_%s", from, to) == 2) {
232                 // 初期読み込み時は "Success" の表示を消すために静かに繋ぐ
233                 struct member_list *from_user = find_user(from);
234                 struct member_list *to_user = find_user(to);
235                 if (from_user && to_user) {
236                     struct follow_node *new_follow = malloc(sizeof(struct
237                         follow_node));
238                     new_follow->user = to_user;
239                     new_follow->next = from_user->follow_head;
240                     from_user->follow_head = new_follow;
241                 }
242             }
243         }
244     }
245     fclose(fp);
246 }

```

```

247 // メニュー表示
248 void print_menu() {
249     printf("---User Management System---\n");
250     printf("Commands:\n");
251     printf("search [name] : Show user info and following list\n");
252     printf("add [name] [age] : Register a new user\n");
253     printf("follow [from] [to] : Create a follow relationship\n");
254     printf("unfollow [from] [to] : Remove a follow relationship\n");
255     printf("is_mutual [name1] [name2] : Check mutual follow status\n");
256     printf("recommend [name] : Show 'friend of friends'\n");
257     printf("exit : Quit program\n");
258     printf("-----\n\n");
259 }
260 // ハッシュの偏りを調べる関数
261 void dist() {
262     printf("---Hash Distribution---\n");
263
264     int total_users = 0; // 全ユーザー数
265
266     // テーブルの端から端まで (0 から TABLE_SIZE - 1 まで) 確認する
267     for (int i = 0; i < TABLE_SIZE; i++) {
268
269         int count = 0; // このバケットに入っている人数
270         struct member_list *current = hashtable[i];
271
272         // 【ここに while ループを書いて、current が NULL になるまでリストを辿りながら count を
273            +1 してください】
274         while(current != NULL){
275             current = current->next;
276             count++;
277         }
278
279         // もし人以上入っていれば、そのバケットの人数を表示する1
280         if (count > 0) {
281             printf("Bucket [%3d]: %d users\n", i, count);
282         }
283
284         total_users += count; // 全体の人数に足し合わせる
285     }
286
287     printf("-----\n");
288     printf("Total users loaded: %d\n", total_users);
289
290     // 理想的な平均人数を計算して表示する
291     // (全体の人数÷ハッシュテーブルのサイズ)
292     double average = (double)total_users / TABLE_SIZE;
293     printf("Ideal average per bucket: %.2f users\n", average);
294 }

```

```

295
296 // 関数main
297 int main() {
298     char input[256];
299     char cmd[32], arg1[64], arg2[64];
300
301     // 配列の初期化安全のため ()
302     for (int i = 0; i < TABLE_SIZE; i++) {
303         hashtable[i] = NULL;
304     }
305
306     // 1. データの読み込み
307     printf("Loading data from data90000.txt...\n");
308     load_data("data90000.txt");
309     printf("Done.\n");
310
311     // 2. メニューの表示
312     print_menu();
313
314     // 3. 対話的なコマンド受付ループ
315     while (1) {
316         printf("command>\n");
317
318         if (fgets(input, sizeof(input), stdin) == NULL) {
319             break;
320         }
321         input[strcspn(input, "\n")] = '\0';
322         if (strlen(input) == 0) continue;
323
324         cmd[0] = '\0'; arg1[0] = '\0'; arg2[0] = '\0';
325         int num_args = sscanf(input, "%s%s%s", cmd, arg1, arg2);
326
327         if (strcmp(cmd, "exit") == 0) {
328             printf("Goodbye!\n");
329             break;
330         }
331         else if (strcmp(cmd, "search") == 0 && num_args >= 2) {
332             search(arg1);
333         }
334         else if (strcmp(cmd, "add") == 0 && num_args >= 3) {
335             add(arg1, atoi(arg2));
336             printf("Success: Added %s.\n", arg1);
337         }
338         else if (strcmp(cmd, "follow") == 0 && num_args >= 3) {
339             follow(arg1, arg2);
340         }
341         else if (strcmp(cmd, "unfollow") == 0 && num_args >= 3) {
342             unfollow(arg1, arg2);
343         }

```

```

344     else if (strcmp(cmd, "is_mutual") == 0 && num_args >= 3) {
345         is_mutual(arg1, arg2);
346     }
347     else if (strcmp(cmd, "recommend") == 0 && num_args >= 2) {
348         recommend(arg1);
349     }
350     else if (strcmp(cmd, "dist") == 0) {
351         dist();
352     }
353     else {
354         printf("Unknown command or missing arguments.\n");
355     }
356
357     printf("\n");
358 }
359
360 return 0;
361 }

```

## 6.3 ハッシュ改良後

Listing 4: ソースコード

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #define TABLE_SIZE 101
6
7  struct member_list {
8      char name[64];
9      int age;
10     struct follow_node *follow_head; // フォローリストの先頭
11     struct member_list *next;        // ハッシュ衝突時の次ノードへのポインタ
12 };
13
14 struct follow_node {
15     struct member_list *user;        // フォロー相手へのポインタ
16     struct follow_node *next;
17 };
18
19 struct member_list *hashtable[TABLE_SIZE];
20
21 // 改良版ハッシュ関数（アルゴリズム：衝突が極めて少ない）DJB2
22 int hash(char *key) {
23     unsigned long hashval = 5381;
24     int c;
25

```

```

26     while ((c = (unsigned char)*key++)) {
27         hashval = ((hashval << 5) + hashval) + c; // hashval * 33 + c
28     }
29     return (int)(hashval % TABLE_SIZE);
30 }
31
32 // ユーザー追加関数
33 void add(char *name, int age) {
34     int x = hash(name);
35     struct member_list *ptr = malloc(sizeof(struct member_list));
36     if (ptr == NULL) {
37         printf("Error: Memory allocation failed.\n");
38         return;
39     }
40
41     ptr->age = age;
42     strcpy(ptr->name, name);
43     ptr->follow_head = NULL;
44
45     // ハッシュテーブルの先頭に挿入 (チェーン法)
46     ptr->next = hashtable[x];
47     hashtable[x] = ptr;
48 }
49
50 // ユーザー検索関数
51 struct member_list *find_user(char *name) {
52     int index = hash(name);
53     struct member_list *current = hashtable[index];
54
55     while (current != NULL) {
56         if (strcmp(current->name, name) == 0) {
57             return current;
58         }
59         current = current->next;
60     }
61     return NULL;
62 }
63
64 // フォロー関係作成関数
65 void follow(char *from, char *to) {
66     struct member_list *from_user = find_user(from);
67     struct member_list *to_user = find_user(to);
68
69     if (from_user == NULL || to_user == NULL) {
70         printf("Error: User not found.\n");
71         return;
72     }
73
74     struct follow_node *new_follow = malloc(sizeof(struct follow_node));

```

```

75     if (new_follow == NULL) {
76         printf("Error: Memory allocation failed.\n");
77         return;
78     }
79
80     new_follow->user = to_user;
81     new_follow->next = from_user->follow_head;
82     from_user->follow_head = new_follow;
83
84     printf("Success: %s followed %s.\n", from, to);
85 }
86
87 // フォロー解除関数
88 void unfollow(char *from, char *to) {
89     struct member_list *from_user = find_user(from);
90     struct member_list *to_user = find_user(to);
91
92     if (from_user == NULL || to_user == NULL) {
93         printf("Error: User not found.\n");
94         return;
95     }
96
97     struct follow_node *current = from_user->follow_head;
98     struct follow_node *prev = NULL;
99
100    while (current != NULL) {
101        if (current->user == to_user) {
102            break;
103        }
104        prev = current;
105        current = current->next;
106    }
107
108    if (current == NULL) {
109        printf("Error: %s is not following %s.\n", from, to);
110        return;
111    }
112
113    if (current == from_user->follow_head) {
114        from_user->follow_head = current->next;
115    } else {
116        prev->next = current->next;
117    }
118
119    free(current);
120    printf("Success: %s unfollowed %s.\n", from, to);
121 }
122
123 // ユーザー情報とフォローリストの表示関数

```

```

124 void search(char *name) {
125     struct member_list *target = find_user(name);
126
127     if (target == NULL) {
128         printf("Error: User not found.\n");
129         return;
130     }
131
132     printf("[User] %s (%d)\n", target->name, target->age);
133     printf("Following:");
134
135     struct follow_node *current = target->follow_head;
136     while (current != NULL) {
137         printf("%s", current->user->name);
138         current = current->next;
139     }
140     printf("\n");
141 }
142
143 // 相互フォロー判定関数
144 void is_mutual(char *name1, char *name2) {
145     struct member_list *user1 = find_user(name1);
146     struct member_list *user2 = find_user(name2);
147
148     if (user1 == NULL || user2 == NULL) {
149         printf("Error: User not found.\n");
150         return;
151     }
152
153     int follow1_to_2 = 0;
154     int follow2_to_1 = 0;
155
156     struct follow_node *current1 = user1->follow_head;
157     while (current1 != NULL) {
158         if (current1->user == user2) {
159             follow1_to_2 = 1;
160             break;
161         }
162         current1 = current1->next;
163     }
164
165     struct follow_node *current2 = user2->follow_head;
166     while (current2 != NULL) {
167         if (current2->user == user1) {
168             follow2_to_1 = 1;
169             break;
170         }
171         current2 = current2->next;
172     }

```

```

173
174     if (follow1_to_2 == 1 && follow2_to_1 == 1) {
175         printf("Yes, %s and %s are mutual friends!\n", name1, name2);
176     } else {
177         printf("No, %s and %s are not mutual friends.\n", name1, name2);
178     }
179 }
180
181 // すでにフォローしているか判定する補助関数
182 int is_following(struct member_list *from, struct member_list *to) {
183     struct follow_node *current = from->follow_head;
184     while (current != NULL) {
185         if (current->user == to) {
186             return 1;
187         }
188         current = current->next;
189     }
190     return 0;
191 }
192
193 // おすすめユーザー推薦関数（既存フォロー排除版）
194 void recommend(char *name) {
195     struct member_list *target = find_user(name);
196     if (target == NULL) {
197         printf("Error: User not found.\n");
198         return;
199     }
200
201     printf("Recommended for %s:\n", name);
202     struct follow_node *current_friend = target->follow_head;
203
204     while (current_friend != NULL) {
205         struct follow_node *current_fof = current_friend->user->follow_head;
206         while (current_fof != NULL) {
207             // 自分自身を除外し、かつすでにフォローしているユーザーも除外する
208             if (current_fof->user != target && is_following(target, current_fof->user) == 0) {
209                 printf("- %s (because %s follows them)\n", current_fof->user->name, current_friend->user->name);
210             }
211             current_fof = current_fof->next;
212         }
213         current_friend = current_friend->next;
214     }
215 }
216
217 // ハッシュ分布表示関数（偏り調査用）
218 void dist() {
219     printf("--- Hash Distribution ---\n");

```

```

220     int total_users = 0;
221
222     for (int i = 0; i < TABLE_SIZE; i++) {
223         int count = 0;
224         struct member_list *current = hashtable[i];
225
226         while (current != NULL) {
227             current = current->next;
228             count++;
229         }
230
231         if (count > 0) {
232             printf("Bucket[%3d]: %d users\n", i, count);
233         }
234         total_users += count;
235     }
236
237     printf("-----\n");
238     printf("Total users loaded: %d\n", total_users);
239     double average = (double)total_users / TABLE_SIZE;
240     printf("Ideal average per bucket: %.2f users\n", average);
241 }
242
243 // ファイル読み込み処理関数
244 void load_data(char *filename) {
245     FILE *fp = fopen(filename, "r");
246     if (fp == NULL) {
247         printf("Error: Cannot open file %s\n", filename);
248         return;
249     }
250
251     char line[256];
252     int mode = 0; // 0: 初期, 1: USERS, 2: FOLLOWS
253
254     while (fgets(line, sizeof(line), fp) != NULL) {
255         line[strcspn(line, "\n")] = 0;
256
257         if (line[0] == '#' || line[0] == '\0') {
258             if (strstr(line, "#USERS")) mode = 1;
259             else if (strstr(line, "#FOLLOWS")) mode = 2;
260             continue;
261         }
262
263         if (mode == 1) {
264             char name[64];
265             int age;
266             if (sscanf(line, "%s %d", name, &age) == 2) {
267                 add(name, age);
268             }

```

```

269     } else if (mode == 2) {
270         char from[64], to[64];
271         if (sscanf(line, "%s%s", from, to) == 2) {
272             struct member_list *from_user = find_user(from);
273             struct member_list *to_user = find_user(to);
274             if (from_user && to_user) {
275                 struct follow_node *new_follow = malloc(sizeof(struct
276                     follow_node));
277                 new_follow->user = to_user;
278                 new_follow->next = from_user->follow_head;
279                 from_user->follow_head = new_follow;
280             }
281         }
282     }
283     fclose(fp);
284 }
285
286 // メニュー表示関数
287 void print_menu() {
288     printf("---User Management System---\n");
289     printf("Commands:\n");
290     printf("search [name] : Show user info and following list\n");
291     printf("add [name] [age] : Register a new user\n");
292     printf("follow [from] [to] : Create a follow relationship\n");
293     printf("unfollow [from] [to] : Remove a follow relationship\n");
294     printf("is_mutual [name1] [name2] : Check mutual follow status\n");
295     printf("recommend [name] : Show 'friend of friends'\n");
296     printf("dist : Show hash distribution status\n");
297     printf("exit : Quit program\n");
298     printf("-----\n\n");
299 }
300
301 // メイン関数
302 int main() {
303     char input[256];
304     char cmd[32], arg1[64], arg2[64];
305
306     for (int i = 0; i < TABLE_SIZE; i++) {
307         hashtable[i] = NULL;
308     }
309
310     printf("Loading data from sns_data.txt...\n");
311     load_data("sns_data.txt");
312     printf("Done.\n");
313
314     print_menu();
315
316     while (1) {

```

```

317     printf("command_>");
318
319     if (fgets(input, sizeof(input), stdin) == NULL) {
320         break;
321     }
322     input[strcspn(input, "\n")] = '\0';
323     if (strlen(input) == 0) continue;
324
325     cmd[0] = '\0'; arg1[0] = '\0'; arg2[0] = '\0';
326     int num_args = sscanf(input, "%s%s%s", cmd, arg1, arg2);
327
328     if (strcmp(cmd, "exit") == 0) {
329         printf("Goodbye!\n");
330         break;
331     }
332     else if (strcmp(cmd, "search") == 0 && num_args >= 2) {
333         search(arg1);
334     }
335     else if (strcmp(cmd, "add") == 0 && num_args >= 3) {
336         add(arg1, atoi(arg2));
337         printf("Success: Added %s.\n", arg1);
338     }
339     else if (strcmp(cmd, "follow") == 0 && num_args >= 3) {
340         follow(arg1, arg2);
341     }
342     else if (strcmp(cmd, "unfollow") == 0 && num_args >= 3) {
343         unfollow(arg1, arg2);
344     }
345     else if (strcmp(cmd, "is_mutual") == 0 && num_args >= 3) {
346         is_mutual(arg1, arg2);
347     }
348     else if (strcmp(cmd, "recommend") == 0 && num_args >= 2) {
349         recommend(arg1);
350     }
351     else if (strcmp(cmd, "dist") == 0) {
352         dist();
353     }
354     else {
355         printf("Unknown command or missing arguments.\n");
356     }
357
358     printf("\n");
359 }
360
361 return 0;
362 }

```